МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 5. Циклические алгоритмы

Студент
гр. БИН-26-03 _____ А. В. Лямин

Студент
гр. БИН-26-03 _____ А. В. Тимченко

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Обратный отсчет

В первом задании требуется организовать цикл с предусловием для вывода чисел в порядке убывания. Пользователь вводит целое неотрицательное число n. Программа должна последовательно вывести все целые числа от n до 0 включительно, уменьшая текущее значение на единицу на каждой итерации.

Алгоритм решения первого задания представлен на рисунке 1.

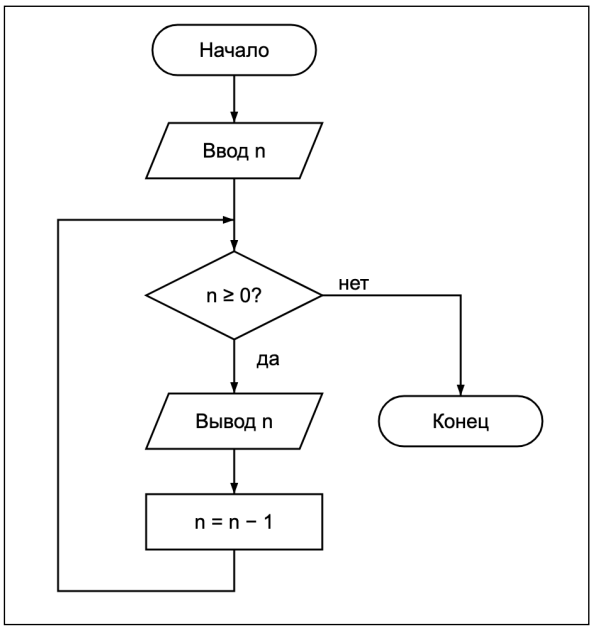


Рисунок 1 – Блок-схема программы для задания 1

На блок-схеме показаны ввод исходного числа n, проверка условия продолжения цикла $n \geq 0$, вывод текущего значения и уменьшение переменной n на единицу. Алгоритм завершается, когда условие цикла становится ложным. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 1 представлен на рисунке 2.

```
n = int(input())
while n >= 0:
    print(n)
    n -= 1
```

Рисунок 2 – Текст программы для задания 1

Программа получает начальное число и с помощью конструкции цикла `while` осуществляет последовательный декремент и вывод чисел.

Результат работы программы при вводе 5 представлен на рисунке 3.

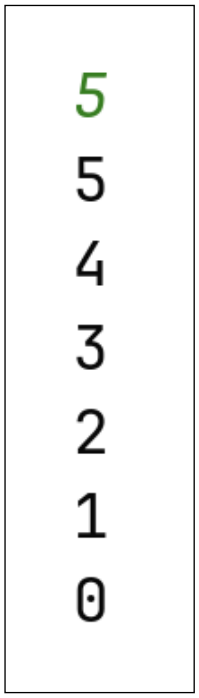


Рисунок 3 – Результат работы программы для задания 1

1.2 Задание 2. Сумма и произведение чисел в диапазоне

Во втором задании требуется вычислить сумму и произведение всех целых чисел на отрезке от a до b включительно. Пользователь вводит два целых числа a и b на отдельных строках ($a \leq b$). Для накопления суммы используется переменная с начальным значением 0, а для произведения — переменная с начальным значением 1.

Алгоритм решения второго задания представлен на рисунке 4.

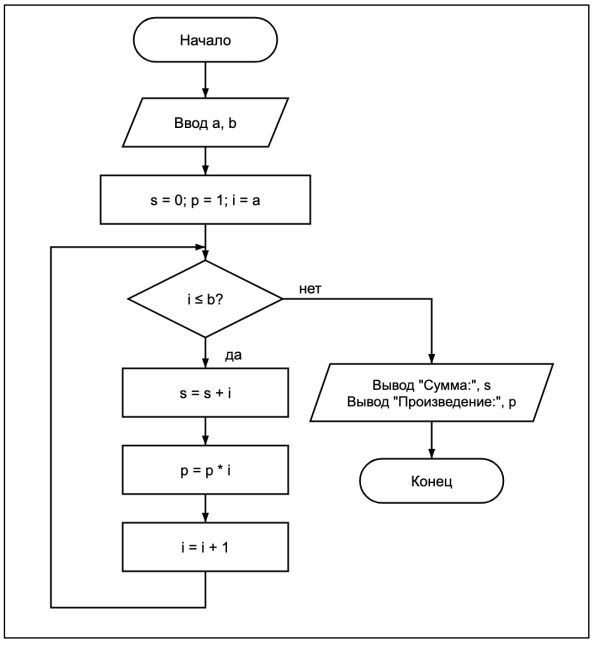


Рисунок 4 – Блок-схема программы для задания 2

На блок-схеме отражены ввод границ диапазона a и b, инициализация накопителей $s = 0$, $p = 1$, параметр цикла $i = a$, проверка условия $i \leq b$, накопление суммы и произведения с последующим шагом $i = i + 1$. По завершении цикла осуществляется вывод итоговых значений. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 2 представлен на рисунке 5.

```
a = int(input())
b = int(input())
s = 0
p = 1
for i in range(a, b + 1):
    s += i
    p *= i
print(f'Сумма: {s}')
print(f'Произведение: {p}')
```

Рисунок 5 – Текст программы для задания 2

Программа выполняет итерацию по числовому диапазону с помощью цикла `for` и функции `range(a, b + 1)`, накапливая промежуточные результаты.

Результат работы программы при вводе 1 и 5 представлен на рисунке 6.

```
1
5
Сумма: 15
Произведение: 120
```

Рисунок 6 – Результат работы программы для задания 2

1.3 Задание 3. Анализ показаний температуры

В третьем задании требуется обработать серию измерений температуры и подсчитать количество значений, попадающих в различные диапазоны. Пользователь сначала вводит целое число n — количество измерений, а затем n вещественных значений температуры на отдельных строках. Программа должна определить число показаний ниже нуля, равных нулю и выше нуля.

Алгоритм проверки и распределения показаний температуры представлен на рисунке 7.

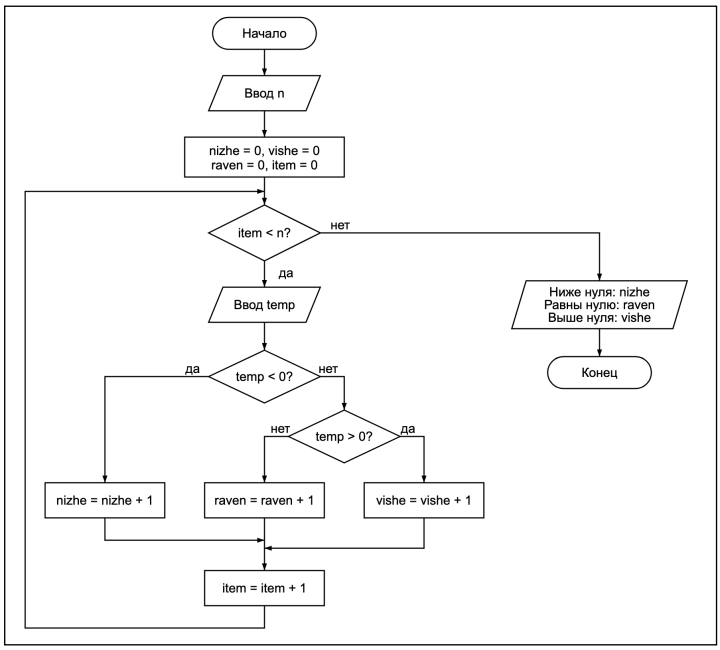


Рисунок 7 – Блок-схема программы для задания 3

На блок-схеме показан цикл с параметром, в теле которого последовательно считываются значения температуры и с помощью условных блоков проверяются условия: отрицательное значение, положительное значение или ноль с инкрементом соответствующего счетчика. По окончании цикла выводятся накопленные счетчики. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 3 представлен на рисунке 8.

```
n = int(input())
nizhe = 0
vishe = 0
raven  = 0
for item in range(n):
    temp = float(input())
    if temp < 0:
        nizhe += 1
    elif temp > 0:
        vishe += 1
    else:
        raven += 1
print(f'Ниже нуля: {nizhe}')
print(f'Равны нулю: {raven}')
print(f'Выше нуля: {vishe}')
```

Рисунок 8 – Текст программы для задания 3

Программа считывает вещественные числа и последовательно обновляет счетчики групп.

Результат работы программы при вводе 5 и последующих значений −2.5, 0, 3.1, −0.5, 4.0 представлен на рисунке 9.

```
5
-2.5
0
3.1
-0.5
4.0
Ниже нуля: 2
Равны нулю: 1
Выше нуля: 2
```

Рисунок 9 – Результат работы программы для задания 3

1.4 Задание 4. Таблица умножения

В четвертом задании требуется вывести таблицу умножения для заданного пользователем целого числа n на множители от 1 до 10 включительно. Каждая строка должна содержать уравнение в формате «$n \cdot i =$ результат».

Алгоритм формирования таблицы умножения представлен на рисунке 10.

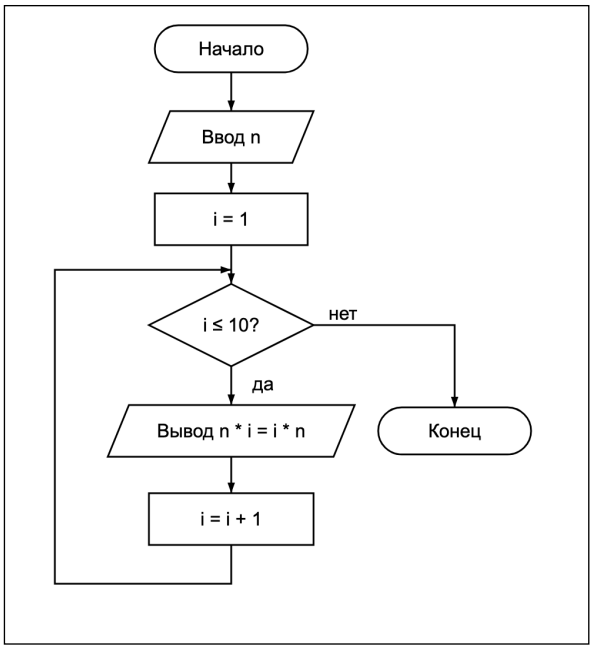


Рисунок 10 – Блок-схема программы для задания 4

На блок-схеме показаны ввод исходного числа n, инициализация переменной цикла $i = 1$, проверка условия $i \leq 10$, вывод произведения $n * i$ и инкремент $i = i + 1$. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 4 представлен на рисунке 11.

```
n = int(input())
for i in range(1, 11):
    print(f'{n} * {i} = {i * n}')
```

Рисунок 11 – Текст программы для задания 4

Для вывода отформатированных строк используется цикл `for` с диапазоном `range(1, 11)` и f-строки.

Результат работы программы при вводе 7 представлен на рисунке 12.

```
7
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
```

Рисунок 12 – Результат работы программы для задания 4

1.5 Задание 5. Расчет стоимости покупок

В пятом задании требуется обработать поток данных о стоимости совершенных покупок без заранее известного их количества. Пользователь вводит целые числа (стоимость каждой покупки) до момента окончания входного потока данных. Программа должна вычислить общее число покупок, суммарную стоимость и среднюю стоимость одной покупки.

Алгоритм обработки покупок представлен на рисунке 13.

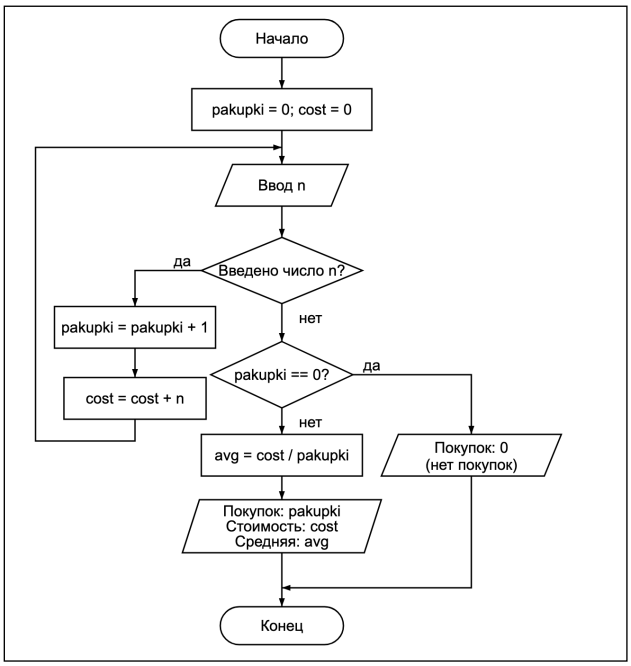


Рисунок 13 – Блок-схема программы для задания 5

На блок-схеме показаны инициализация счетчика покупок и суммарной стоимости, цикл чтения чисел с обработкой исключения конца ввода, расчет среднего значения и вывод итоговых статистических данных. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 5 представлен на рисунке 14.

```
pakupki = 0
cost = 0
while True:
    try: n = int(input())
    except: break
    pakupki += 1
    cost += n


print(f'Покупок: {pakupki}')
print(f'Общая стоимость: {cost}')
print(f'Средняя стоимость: {cost / pakupki}')
```

Рисунок 14 – Текст программы для задания 5

Цикл `while True` обрабатывает ввод до прерывания, перехватываемого блоком `try...except`, после чего рассчитываются и выводятся статистика и среднее значение.

Результат работы программы при вводе 120, 350, 280, 150 и 100 представлен на рисунке 15.

```
120
350
280
150
100
Покупок: 5
Общая стоимость: 1000
Средняя стоимость: 200.0
```

Рисунок 15 – Результат работы программы для задания 5

1.6 Задание 6. Треугольник Паскаля

В шестом задании требуется сгенерировать первые n строк арифметического треугольника Паскаля, где каждый элемент строки вычисляется через биномиальные коэффициенты, и подсчитать суммарное количество нечетных элементов во всей построенной числовой пирамиде. Количество строк n вводится пользователем.

Алгоритм построения треугольника Паскаля и подсчета нечетных элементов представлен на рисунке 16.

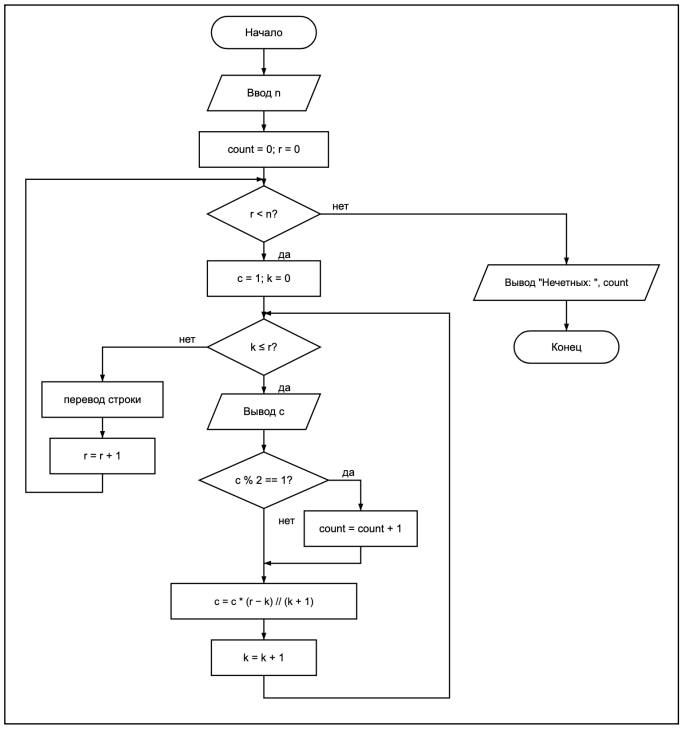


Рисунок 16 – Блок-схема программы для задания 6

На блок-схеме отражена организация двух вложенных циклов: внешний цикл перебирает строки от 0 до $n - 1$, а внутренний цикл вычисляет коэффициенты строки, проверяет их на нечетность (`c % 2 == 1`), накапливает счетчик нечетных чисел и рассчитывает следующий

элемент. После завершения циклов выводится суммарное число нечетных элементов. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 6 представлен на рисунке 17.

```
n = int(input())
count = 0
for r in range(n):
    c = 1
    for k in range(r + 1):
        print(c, end=" ")
        if c % 2 == 1:
            count += 1
        c = c * (r - k) // (k + 1)
    print()
print("Нечетных элементов:", count)
```

Рисунок 17 – Текст программы для задания 6

Программа использует формулу $C_n^{k+1} = C_n^k \cdot \frac{n-k}{k+1}$ для вычисления элементов строки без прямого подсчета факториалов, проверяя каждый элемент на нечетность.

Результат работы программы при вводе 5 представлен на рисунке 18.

```
5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
Нечетных элементов: 11
```

Рисунок 18 – Результат работы программы для задания 6